GREEDY ALGORITHM PROBLEMS

1. Coin Change Problem

```
#include <stdio.h>

int main() {
```

```
int amount;

if (scanf("%d", &amount) != 1) return 0;

int denominations[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1}; int

n = sizeof(denominations) / sizeof(denominations[0]); int

total_count = 0;

for (int i = 0; i < n; i++) {

if (amount == 0) {

break;

}

total_count += amount / denominations[i];

amount %= denominations[i];

}

printf("%d\n", total_count);

return 0;

}
```

2. Assigning Cookies Problem

```
#include <stdio.h>

#include <stdlib.h>

int compare(const void *a, const void *b) {
```

```
return (*(int *)a - *(int *)b);

}

int main() {

int n;

if (scanf("%d", &n) != 1) return 0;

int *g = (int *)malloc(n * sizeof(int));

for (int i = 0; i < n; i++) {

scanf("%d", &g[i]);

}

int m;

if (scanf("%d", &m) != 1) return 0;

int *s = (int *)malloc(m * sizeof(int));

for (int j = 0; j < m; j++) {

scanf("%d", &s[j]);

}

qsort(g, n, sizeof(int), compare);

qsort(s, m, sizeof(int), compare);

int child_ptr = 0;

int cookie_ptr = 0;
```

```
while (child_ptr < n && cookie_ptr < m) {

if (s[cookie_ptr] >= g[child_ptr]) {

child_ptr++;

}

cookie_ptr++;

}

printf("%d\n", child_ptr);

free(g);

free(s);

return 0;

}
```

3. Maximum Burger Selection Problem

```
#include <stdio.h>

#include<math.h>

int main()

{

int n;

scanf("%d",&n);

int arr[n];
```

```
for(int i=0;i<n;i++)
{
scanf("%d",&arr[i]);
}
for(int i=0;i<n-1;i++)
{
for(int j=0;j<n-i-1;j++)
{
if(arr[j]<arr[j+1])
{
int temp=arr[j];
arr[j]=arr[j+1];
arr[j+1]=temp;
}
}
}
int s=0;
for(int i=0;i<n;i++)
{
```

```
for (int i = 0; i < n; i++) {

}

printf("%d",s);

}
```

4. Maximum Weighted Array Sum

```
#include <stdio.h>

#include <stdlib.h>



int cmp(const void *a, const void

*b) {

   return (*(int *)a > *(int *)b) -

(*(int *)a < *(int *)b);

}



int main() {

if (int_a > int_b) return 1;
```

```
for (int i = 0; i < n; i++) {
    int n;

    scanf("%d", &n);




    int a[n];

    for (int i = 0; i < n; i++)

        scanf("%d", &a[i]);




    qsort(a, n, sizeof(int), cmp);




    long long sum = 0;

    for (int i = 0; i < n; i++)

        sum += (long long)a[i] * i;




    printf("%lld", sum);
if (int_a > int_b) return 1;
```

```
for (int i = 0; i < n; i++) {


   return 0;

}
```

5. Minimum Product of Array Elements

```
#include <stdio.h>


#include <stdlib.h>

int compare_asc(const void *a, const void *b) {

long long int_a = *(long long *)a;

long long int_b = *(long long *)b;









if (int_a > int_b) return 1;
```

```
if (int_a < int_b) return -1;

return 0;

}

int main() {

int n;

if (scanf("%d", &n) != 1) return 0;

long long *a = (long long *)malloc(n * sizeof(long long));

long long *b = (long long *)malloc(n * sizeof(long long));

if (!a || !b) return 0;

for (int i = 0; i < n; i++) {

scanf("%lld", &a[i]);

}

for (int i = 0; i < n; i++) {

scanf("%lld", &b[i]);

}

qsort(a, n, sizeof(long long), compare_asc);

qsort(b, n, sizeof(long long), compare_asc);

long long min_sum = 0;

for (int i = 0; i < n; i++) {
```

```
min_sum += a[i] * b[n - 1 - i];

}

printf("%lld\n", min_sum);

free(a);

free(b);

return 0;

}
```